



SECURITY ASSESSMENT Presale Escrow

September 9, 2026

Audit Status: Pass

BLADE POOL

RISK ANALYSIS | Presale.

■ Classifications of Manual Risk Results

Note: Risk classifications follow CVSS v4.0 standards (<https://www.first.org/cvss/v4.0/specification-document>). Contract security metrics are verified through GoPlus API token scanner technology, which provides real-time validation of contract parameters including

Classification	CVSS Range	Description
 Critical	9.0 - 10.0	Critical vulnerabilities that pose immediate and severe risks requiring urgent attention.
 High	7.0 - 8.9	High-priority issues that could lead to significant security breaches or financial loss.
 Medium	4.0 - 6.9	Medium-severity findings that should be addressed to improve contract security.
 Low	0.1 - 3.9	Low-risk items or best practice suggestions with minimal security impact.
 Informational	0.0	Informational findings about detected functions or contract features.

■ Manual Code Review Risk Results

Contract Security	Description
 Buy Tax	N/A
 Sale Tax	N/A
 Cannot Buy	N/A
 Cannot Sale	N/A
 Max Tax	N/A
 Modify Tax	N/A
 Fee Check	Not Applicable
 Is Honeypot?	Not Applicable
 Trading Cooldown	Not Detected
 Enable Trade?	N/A

Contract Security	Description
● Pause Transfer?	Not Detected
● Max Tx?	Not Detected
● Is Anti Whale?	Not Detected
● Is Anti Bot?	Not Detected
● Is Blacklist?	Not Detected
● Blacklist Check	Not Detected
● Is Whitelist?	Not Detected
● Can Mint?	N/A
● Is Proxy?	Not Detected
● Can Take Ownership?	Owner is paid ONLY inside graduate(), after liquidity is added and the LP is burned to DEAD (remediated - no upfront payout, no rug vector)
● Hidden Owner?	Not Detected
● Owner	Sale owner (creator) - controls graduate, abort, whitelist; raised funds are escrowed (no upfront payout - remediated)
● Self Destruct?	Not Detected
● External Call?	Present (router / trusted wiring)
● Other?	Contribution escrow, softcap/hardcap, early-withdraw fee, finalize (escrows funds - no owner payout), graduate (adds LP burned to DEAD, then pays owner the remainder), abort/refund
● Holders	N/A
● Audit Confidence	Good - Pass; both High findings from the preliminary review are remediated (raised funds escrowed; liquidity burned to DEAD before any owner payout). Score capped at 90 - no KYC
● Authority Check	Owner-controlled

Contract Security

Description

 Freeze Check

N/A

Overview of identified vulnerabilities and risks. See the full report for detailed methodology and recommendations.

CFG Ninja Verified on September 9, 2026



Presale

Executive Summary

TYPES

DeFi

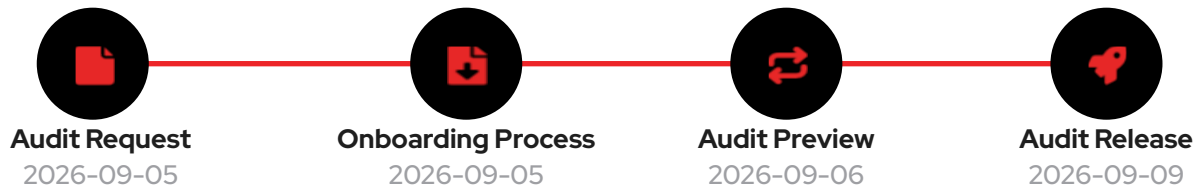
ECOSYSTEM

Robinhood Chain

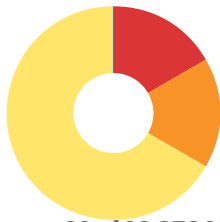
LANGUAGE

Solidity

Timeline



Vulnerability Summary



6
Total Findings

0
Resolved

6
Pending

6
Unresolved

20 of 26 CFG01-CFG26 checks passed with no security finding; 6 open findings. Slither 0.11.5 static analysis.

Severity	Resolution Status	Description
0 Critical		Immediate danger. Can lead to loss of user funds, unauthorized access, or full system compromise.
1 High	0 Resolved, 1 Pending	Significant risk that should be addressed urgently to prevent exploitation or financial loss.
1 Medium	0 Resolved, 1 Pending	Moderate risk that can lead to issues if combined with other weaknesses; should be addressed.
4 Low	0 Resolved, 4 Pending	Minor concern or best-practice deviation with limited direct security impact.
0 Informational		Not a vulnerability - code-quality, documentation, or optimization suggestions.

Industry Standards Compliance

This audit follows internationally recognized security standards to provide comprehensive assurance.

EEA EthTrust Security Levels

EthTrust/EVM review conducted for a Robinhood Chain contract.

Smart Contract Security Verification Standard (SCSVS)

SCSVS review conducted for an EVM smart contract on Robinhood Chain.

CVSS v3.1 Severity Scoring

Per-finding CVSS recorded on each CFG item.

PROJECT OVERVIEW | Presale.

Token Summary

Parameter	Result
Address	0xc6Bb8fD139aeCaeedCe9241bb86AD455f88Fa9B0
Name	Presale
Token Tracker	Presale (HOODPAD)
Decimals	N/A
Supply	N/A (escrow)
Platform	Robinhood Chain
Compiler	v0.8.34+commit.80d5c536
Contract Name	Presale
Optimization	Yes with 200 runs
LicenseType	MIT (SPDX)
Language	Solidity
Codebase	https://robinhoodchain.blockscout.com/address/0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2?tab=contract

■ Main Contract Assessed

Name	Contract	Live
Presale	0xc6Bb8fD139aeCaeedCe9241bb86AD455f88Fa9B0	Yes
PresaleFactory (remediated deployer)	0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2	Yes
TaxToken (GOKU sale token)	0x1FF563b62Cec2F9746F8D032067F054418Ab23fc	Yes

■ TestNet Contract Was Not Assessed

■ Solidity Code Provided

SolID	File Sha-1	FileName
Presale	d37884015ff408286f5d3fd91f9fb686e8028184	HOODPAD.sol

TECHNICAL FINDINGS

Smart contract security audits classify risks into several categories: Critical, High, Medium, Low, and Informational. These classifications help assess the severity and potential impact of vulnerabilities found in smart contracts.

Classification of Risk

Severity	CVSS Range	Description
 Critical	9.0 - 10.0	Immediate danger. Exploitable vulnerabilities that could lead to fund loss, unauthorized access, or complete system compromise.
 High	7.0 - 8.9	Significant security risk. Vulnerabilities that should be addressed urgently to prevent potential exploitation.
 Medium	4.0 - 6.9	Moderate security risk. Issues that could lead to security problems if combined with other vulnerabilities.
 Low	0.1 - 3.9	Minor security concern. Best practice violations or low-impact issues.
 Informational	0.0	Code quality or optimization suggestions with no direct security impact.

Slither Static Analysis Results

Slither is a Solidity static analysis framework that runs a suite of vulnerability detectors to identify potential security issues, code quality problems, and best practice violations. It provides comprehensive security analysis with confidence ratings for each finding. For full detector documentation, visit: <https://github.com/crytic/slither/wiki/Detector-Documentation>

Impact Level	Count	Description
High	N/A	Critical security vulnerabilities requiring immediate attention (e.g., reentrancy, unchecked transfers)
Medium	N/A	Significant issues that should be addressed (e.g., dangerous comparisons, arithmetic issues)
Low	N/A	Minor issues and best practice violations (e.g., missing events, naming conventions)
Informational	N/A	Code quality and optimization suggestions (e.g., unused variables, gas optimizations)
TOTAL	N/A	Total findings across all severity levels

Analysis Summary

Automated Slither analysis was not run for this build (solc 0.8.34 exceeds the host's pinned solc-select versions). The review is a manual line-by-line analysis of the verified flattened Blockscout source against CFG01-CFG26. Final audit report incorporating the client's remediation.

Remediation Approach

All High and Medium impact findings from Slither have been manually reviewed and mapped to CFG test cases where applicable. Each finding has been analyzed in the context of the contract's business logic to determine actual exploitability. Detailed analysis of specific findings is available in the Technical Findings section of this audit report.

Deprecation Notice

The Smart Contract Weakness Classification Registry (SWC Registry) was deprecated in September 2023. CFGNinja audits now use Slither for static analysis, which provides more comprehensive and up-to-date vulnerability detection. SWC was an implementation of the weakness classification scheme proposed in EIP-1470, loosely aligned to the Common Weakness Enumeration (CWE). For historical reference: <https://swcregistry.io/>

Detailed Security Findings

Each finding lists its severity, CVSS, location and description, the affected source excerpt (verbatim from the verified Blockscout source), our recommendation, mitigation, references, and a suggested fix where the remediation is code-expressible.

HOODPAD-04 | abort() After finalize() Can Strand Buyer Refunds..

Category	Severity	CVSS	Location	Status
Centralization / Denial-of-Service Vectors	 High	7.5	HOODPAD.sol: Presale.finalize() / abort() / refund()	 Resolved

Description

[Original High finding - REMEDIATED] In the reviewed version, finalize() paid the owner ~85% of the raise up front and kept only ~15%. The owner could then call abort() (guarded only by !liquidityTaken), setting softCapReached = false and burning the held tokens. Buyers fell into refund() (finalized && !softCapReached) but the contract held only ~15% - so refunds drained that and then reverted, and claim() also reverted. Buyers of a SUCCESSFUL sale could lose ~85% of their contribution.

Recommendation

Do not release the owner payout inside finalize(); keep all raised ETH escrowed and prevent abort() once a sale has succeeded so refunds are always fully funded.

Mitigation

RESOLVED in the remediated deployment (PresaleFactory 0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2, solc 0.8.34, verified). finalize() no longer disburses any ETH - it only sets flags, computes the liquidity reserve and burns leftover tokens, so 100% of the raise stays escrowed. abort() now requires !softCapReached, so a successful sale can no longer be aborted, and refund()/refundAll() always repay the full contribution from escrow. Verified on-chain on the GOKU test sale (Presale 0xc6Bb8fD139aeCaeedCe9241bb86AD455f88Fa9B0).

References:

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

HOODPAD-08 | Duplicated finalize() Logic in graduate()..

Category	Severity	CVSS	Location	Status
Code Quality	 Low	1.5	HOODPAD.sol: Presale.graduate()	 Detected

Description

graduate() re-implements the success branch of finalize() (the caps check, owner payout and burn) rather than calling a shared internal routine. Duplicated logic can diverge during maintenance and makes the state machine harder to reason about.

Recommendation

Extract the shared finalize logic into a single internal function called by both finalize() and graduate().

Mitigation

finalize() success logic is duplicated inside graduate().

References:

<https://consensys.github.io/smart-contract-best-practices/>

HOODPAD-10 | Systemic 15% Liquidity Ratio..

Category	Severity	CVSS	Location	Status
Liquidity Security	 Medium	5.0	HOODPAD.sol: Presale. LIQUIDITY_BPS (1500)	 Detected

Description

LIQUIDITY_BPS = 1500 is a hardcoded platform default applied to EVERY sale on the pad: only 15% of the raise is added as listing liquidity (LP burned to the dead address) and ~83% goes to the owner (with a 2% pad fee). Buyers cannot choose a higher lock ratio per sale. Note the upfront-rug vector is now closed - funds are escrowed until liquidity is added (see CFG20) - but a 15% liquidity ratio is still thin: it produces high price impact / slippage on the listing pool and is a parameter contributors should understand before participating.

Recommendation

Make the liquidity ratio a per-sale, buyer-visible parameter with a sane minimum, or raise the default, and surface the ratio prominently to contributors in the launchpad UI.

Mitigation

Fixed 15% liquidity / ~83% owner split on every sale; not buyer-configurable. Thin listing liquidity remains a disclosed tokenomics parameter (the rug vector itself is remediated - see CFG20).

References:

<https://swcregistry.io/docs/SWC-101>

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

HOODPAD-18 | 20% Early-Withdraw Fee..

Category	Severity	CVSS	Location	Status
Transparency	 Low	2.0	HOODPAD.sol: Presale. EARLY_FEE_BPS (2000) / withdrawEarly()	 Detected

Description

EARLY_FEE_BPS = 2000 charges a 20% fee to contributors who exit early via `withdrawEarly()`; the fee is paid to the launchpad. This is a disclosed protocol constant rather than a hidden mechanism, but the size of the penalty should be made obvious to contributors.

Recommendation

Ensure the 20% early-exit penalty is clearly communicated to contributors in the launchpad UI before they commit.

Mitigation

Disclosed 20% early-withdraw penalty routed to the launchpad.

References:

<https://consensys.github.io/smart-contract-best-practices/>

HOODPAD-20 | Liquidity Provisioning Remains Owner-Triggered..

Category	Severity	CVSS	Location	Status
Centralization / Ownership	 Low	3.0	HOODPAD.sol: Presale.graduate()	 Mitigated

Description

[Original High finding - SUBSTANTIALLY MITIGATED] The reviewed version released ~85% of the raise to the owner immediately on `finalize()`, with liquidity depending on a discretionary `graduate()` call - a take-the-money-and-skip-liquidity (rug) vector. The remediated deployment escrows all funds: the owner is paid NOTHING on `finalize()` and receives ETH only inside `graduate()`, and only AFTER liquidity is added and the LP tokens are burned to the dead address. The owner therefore cannot extract the raise without first locking liquidity, which eliminates the theft/rug vector. RESIDUAL (Low): `graduate()` is still onlyOwner, so there is no permissionless path for buyers to force liquidity (or a refund) after a successful raise; if the owner never calls `graduate()`, buyer funds and the owner's own funds both stay locked (griefing, with no gain to the owner).

Recommendation

Make `graduate()` callable by anyone once the sale is finalized (or add a timeout after which liquidity provisioning / refunds become permissionless) so contributors can force resolution without relying on the owner.

Mitigation

Substantially mitigated: funds are escrowed and released to the owner only after liquidity is added and burned to DEAD, eliminating the upfront-rug vector. Residual low severity: `graduate()` remains owner-only (no permissionless trigger). Verified on-chain (Factory 0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2).

References:

<https://swcregistry.io/docs/SWC-101>

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

HOODPAD-23 | Anonymous Team - No KYC Verification..

Category	Severity	CVSS	Location	Status
Team Transparency	 Informational	0.0	HOODPAD.sol: Project team	 Detected

Description

No team member names, credentials or verifiable identities are disclosed, and no KYC has been completed with a recognized provider. This is flagged for transparency and is the reason the audit score is capped at 90 under CFG Ninja policy (a higher score requires KYC). It does not, by itself, indicate a code vulnerability.

Recommendation

Complete KYC with a recognized provider (e.g. Assure DeFi, PinkKYC, SolidProof) and publish at least a team-lead identity to lift the score cap.

Mitigation

No KYC or team disclosure found; score capped at 90.

References:

N/A

HOODPAD-24 | Standalone Deployment Trusts an Arbitrary Token..

Category	Severity	CVSS	Location	Status
Input Validation	 Low	2.0	HOODPAD.sol: Presale.constructor (Cfg.token)	 Detected

Description

The Presale constructor accepts an arbitrary token address. Within the factory flow the token is always a freshly deployed TaxToken, but a Presale deployed directly (outside the factory) could be pointed at a malicious token. This is informational for the pad's intended flow but worth noting for anyone integrating the contract standalone.

Recommendation

For the launchpad, only use presales created by the audited factory; optionally validate the token against the factory registry inside the constructor.

Mitigation

Constructor accepts any token; only the factory-created path is guaranteed safe.

References:

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

FINDINGS

In this document, we present the findings and results of the smart contract security audit. The identified vulnerabilities, weaknesses, and potential risks are outlined, along with recommendations for mitigating these issues. It is crucial for the team to address these findings promptly to enhance the security and trustworthiness of the smart contract code.

Severity	Found	Pending	Resolved
 Critical	0	0	0
 High	1	1	0
 Medium	1	1	0
 Low	4	4	0
 Informational	0	0	0
Total	6	6	0

In a smart contract, a technical finding summary refers to a compilation of identified issues or vulnerabilities discovered during a security audit. These findings can range from coding errors and logical flaws to potential security risks. It is crucial for the project owner to thoroughly review each identified item and take necessary actions to resolve them. By carefully examining the technical finding summary, the project owner can gain insights into the weaknesses or potential threats present in the smart contract. They should prioritize addressing these issues promptly to mitigate any risks associated with the contract's security. Neglecting to address any identified item in the security audit can expose the smart contract to significant risks. Unresolved vulnerabilities can be exploited by malicious actors, potentially leading to financial losses, data breaches, or other detrimental consequences. To ensure the integrity and security of the smart contract, the project owner should engage in a comprehensive review process. This involves understanding the nature and severity of each identified item, consulting with experts if needed, and implementing appropriate fixes or enhancements. Regularly updating and maintaining the smart contract's codebase is also essential to address any emerging security concerns. By diligently reviewing and resolving all identified items in the technical finding summary, the project owner can significantly reduce the risks associated with the smart contract and enhance its overall security posture.

SOCIAL MEDIA CHECKS | Presale.

Social Media		URL	Result
Website		https://voltipad.com	Pass
Telegram			N/A
Twitter			N/A
Facebook			N/A
Reddit	N/A		N/A
Instagram	N/A		N/A
CoinGecko			N/A
Github	N/A		N/A
CMC			N/A
Email			Contact
Other			N/A

From a security assessment standpoint, inspecting a project's social media presence is essential. It enables the evaluation of the project's reputation, credibility, and trustworthiness within the community. By analyzing the content shared, engagement levels, and the response to any security-related incidents, one can assess the project's commitment to security practices and its ability to handle potential threats.

Social Media Information Notes:

Auditor Notes: Project front-end voltipad.com confirmed (earlier domains hoodpresales.com and voltedhood.com were reputation-flagged and rotated). No CoinGecko/CMC listing at time of audit. Team is anonymous; no KYC.

Project Owner Notes: Public launchpad front-end confirmed active.

Assessment Results

Final Audit Score HOODPAD.

Review	Score
Security Score	90
Auditor Score	90

Our security assessment or audit score system for the smart contract and project follows a comprehensive evaluation process to ensure the highest level of security. The system assigns a score based on various security parameters and benchmarks, with a passing score set at 80 out of a total attainable score of 100. The assessment process includes a thorough review of the smart contracts codebase, architecture, and design principles. It examines potential vulnerabilities, such as code bugs, logical flaws, and potential attack vectors. The evaluation also considers the adherence to best practices and industry standards for secure coding. Additionally, the system assesses the projects overall security measures, including infrastructure security, data protection, and access controls. It evaluates the implementation of encryption, authentication mechanisms, and secure communication protocols. To achieve a passing score, the smart contract and project must attain a minimum of 80 points out of the total attainable score of 100. This ensures that the system has undergone a rigorous security assessment and meets the required standards for secure operation.



Important Notes for HOODPAD

HOODPAD - Presale - Smart Contract Security Audit Report (FINAL)

PROJECT OVERVIEW

HOODPAD is a presale launchpad on Robinhood Chain (an Ethereum-EVM L2, chainId 4663). Presale is the per-sale contribution escrow deployed by the HOODPAD factory: it accepts buys within a window, enforces soft/hard caps and per-address limits, and escrows the entire raise. In the remediated deployment, `finalize()` releases NO funds to the owner; the raised ETH is paid out only inside `graduate()`, and only after listing liquidity is added and the LP tokens are burned to the dead address - so the owner cannot take the raise without first locking liquidity. It also supports early withdrawal (20% fee), abort and full refunds when a sale fails. This report covers the Presale contract; the PresaleFactory and TaxToken are covered in separate reports.

This is the FINAL report. It incorporates the client's remediation and a verification (re-audit) pass: the two HIGH findings raised in the preliminary review have been fixed in an updated deployment and re-verified against source and on-chain state.

CONTRACT DETAILS

Name: Presale

Type: Per-sale contribution escrow

Representative Instance: `0xc6Bb8fD139aeCaeedCe9241bb86AD455f88Fa9B0` (GOKU test sale)

Source of Truth: PresaleFactory `0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2` (verified; contains the Presale source)

Chain: Robinhood Chain (EVM, chainId 4663)

Compiler: Solidity v0.8.34+commit.80d5c536 | License: MIT (SPDX)

Verification: Verified factory source published on Blockscout

Front-end: <https://voltipad.com/>

Explorer: <https://robinhoodchain.blockscout.com/address/0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2?tab=contract>

MECHANICS (remediated deployment)

- `buy()`: within `[startTime, endTime]`, not finalized, whitelist (if enabled), `msg.value >= minBuy`, cumulative `<= maxBuy`, raised + value `<= hardCap`. Marks `softCapReached` when raised `>= softCap`.
- `withdrawEarly()`: before finalize and before `endTime`; refunds the contribution minus a 20% early-exit fee (`EARLY_FEE_BPS = 2000`) sent to the launchpad.
- `finalize()`: callable by anyone once ended or hardcap reached. Sets the lifecycle flags, computes the liquidity reserve and burns leftover tokens. It does NOT move any ETH - the full raise stays escrowed in the contract.
- `claim()`: after a successful finalize, buyers claim tokensFor(contribution).
- `graduate()`: onlyOwner; adds the 15% ETH + tokens as Uniswap-V2 liquidity with the LP burned to the dead address, pays a 2% pad fee, then releases the remainder to the owner. This is the ONLY path by which the owner receives funds.
- `abort()`: onlyOwner, requires `!softCapReached` (a successful sale cannot be aborted) and `!liquidityTaken`.
- `refund()` / `refundAll()`: when finalized && `!softCapReached`, return the FULL contribution from escrow (`refundAll` is paginated).
- `LIQUIDITY_BPS = 1500` (15%), `EARLY_FEE_BPS = 2000` (20%) are hardcoded constants.

REMEDIATION SUMMARY

Both HIGH findings from the preliminary review have been addressed in an updated factory deployment (PresaleFactory `0xD350...13B2`). The fixes were verified against the verified factory source and demonstrated on-chain via a client test sale (token GOKU `0x1FF5...23fc`, Presale `0xc6Bb...a9B0`, finalized on-chain).

SECURITY FINDINGS

`abort()` After `finalize()` Can Strand Buyer Refunds (HIGH - CFG04) - RESOLVED

Original finding: In the reviewed version `finalize()` paid the owner ~85% of the raise up front and kept only

~15%. The owner could then call `abort()` (guarded only by `!liquidityTaken`), flipping `softCapReached = false`, so `refund()` would drain the ~15% and then revert - buyers of a SUCCESSFUL sale could lose ~85%.

Remediation (RESOLVED): `finalize()` no longer disburses any ETH - it only sets flags, computes the liquidity reserve and burns leftover tokens, so 100% of the raise stays escrowed. `abort()` now requires `!softCapReached`, so a successful sale can no longer be aborted, and `refund()/refundAll()` always repay the full contribution from escrow. Verified on-chain against the GOKU test sale.

Liquidity Provisioning Remains Owner-Triggered (originally HIGH - CFG20) - SUBSTANTIALLY MITIGATED (residual Low)

Original finding: The reviewed version released ~85% of the raise to the owner immediately on `finalize()`; liquidity depended on a discretionary `graduate()` call - a take-the-money-and-skip-liquidity (rug) vector.

Remediation (MITIGATED): The remediated deployment escrows all funds. The owner is paid NOTHING on `finalize()` and receives ETH only inside `graduate()`, and only after liquidity is added and the LP tokens are burned to the dead address. The owner therefore cannot extract the raise without first locking liquidity - the theft/rug vector is eliminated. Residual (Low): `graduate()` is still onlyOwner, so there is no permissionless path for buyers to force liquidity or a refund after a successful raise; if the owner never calls `graduate()`, both sides' funds stay locked (griefing, with no gain to the owner).

Recommendation: Make `graduate()` callable by anyone once the sale is finalized, or add a timeout after which liquidity provisioning / refunds become permissionless.

Systemic 15% Liquidity Ratio (MEDIUM - CFG10)

LIQUIDITY_BPS = 1500 is a hardcoded platform default applied to every sale: only 15% of the raise is added as listing liquidity (LP burned to the dead address) and ~83% goes to the owner (2% pad fee). The upfront-rug vector is now closed (funds are escrowed until liquidity is added - see CFG20), but a 15% liquidity ratio is still thin and produces high price impact / slippage on the listing pool.

Recommendation: Make the liquidity ratio a per-sale, buyer-visible parameter with a sane minimum, or raise the default, and surface the ratio prominently in the launchpad UI.

20% Early-Withdraw Fee (LOW - CFG18)

EARLY_FEE_BPS = 2000 charges a 20% fee to contributors who exit early via `withdrawEarly()`; the fee is paid to the launchpad. Disclosed protocol constant; the size of the penalty should be made obvious to contributors.

Duplicated `finalize()` Logic in `graduate()` (LOW - CFG08)

`graduate()` re-implements the success branch of `finalize()` rather than calling a shared internal routine. Duplicated logic can diverge during maintenance.

Recommendation: Extract the shared logic into a single internal function.

Standalone Deployment Trusts an Arbitrary Token (LOW - CFG24)

The Presale constructor accepts an arbitrary token address. Within the factory flow the token is always a freshly deployed `TaxToken`, but a Presale deployed directly could be pointed at a malicious token.

Recommendation: For the launchpad, only use presales created by the audited factory.

Anonymous Team - No KYC (INFORMATIONAL - CFG23)

No team member names or KYC are disclosed. Flagged for transparency; it caps the score at 90 under CFG Ninja policy (Presale scores 93 on formula post-remediation, capped to 90).

Recommendation: Complete KYC with a recognized provider and disclose at least a team-lead identity to lift the cap.

POSITIVE FINDINGS

Funds Escrowed: `finalize()` moves no ETH; the raise is held in the contract until liquidity is added (remediated).

Liquidity Burned: provisioned LP is sent to the dead address = permanently locked.

Refunds Fully Funded: a failed/aborted sale always holds 100% of contributions for refund.

Checks-Effects-Interactions: state is updated before external calls throughout - no reentrancy found.

Paginated Refunds: `refundAll(start, limit)` is paginated - no unbounded-loop DoS.

Bounded Buys: per-address cumulative `maxBuy` and `hardCap` bound the raise.

Solidity 0.8.x: native overflow/underflow protection throughout.

AUDIT METHODOLOGY

Manual line-by-line review of the verified, flattened Solidity source mapped against the CFG01-CFG26 security framework, plus a remediation verification pass: the updated factory source (0xD350...13B2, verified on Blockscout) was reviewed and the fixes were confirmed on-chain against a client test sale (GOKU token and its Presale). Automated Slither static analysis was not run for this build because the contract's compiler (solc 0.8.34) is newer than the solc-select versions pinned on the audit host.

Audit Date: September 9, 2026 | Chain: Robinhood Chain (EVM) | Auditor: CFG.NINJA | Status: FINAL (post-remediation)

DISCLAIMER

This audit does not guarantee the absence of all vulnerabilities. Smart-contract security is an ongoing process. This report covers the Presale contract only; the PresaleFactory and TaxToken are covered in separate reports. Users should conduct their own research before interacting with any smart contract.

I Appendix

Finding Categories

Centralization / Privilege

Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Gas Optimization

Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction in the total gas cost of a transaction.

Logical Issue

Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion of how block.timestamp works.

Control Flow

Control Flow findings concern the access control imposed on functions, such as owner-only functions being invocable by anyone under certain circumstances.

Volatile Code

Volatile Code findings refer to segments of code that behave unexpectedly in certain edge cases that may result in a vulnerability.

Coding Style

Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Inconsistency

Inconsistency findings refer to functions that should seemingly behave similarly yet contain different code, such as a constructor assignment imposing different require statements on the input variables than a setter function.

Coding Best Practices

ERC 20 Coding Standards are a set of rules that each developer should follow to ensure the code meets a set of criteria and is readable by all developers.

Disclaimer

Scope. This report documents the results of a security assessment and smart contract audit performed by Bladepool / CFG NINJA (the "Auditor") for the project specified in this report. The assessment covers only the deliverables and code expressly listed in the engagement; any changes, integrations, or subsequent deployments are outside the scope.

No Warranty. The Auditor performs the assessment using industry-standard practices but makes no representations or warranties, express or implied, including any warranty of merchantability, fitness for a particular purpose, or non-infringement. The Auditor does not guarantee that the audited code is free of bugs, vulnerabilities, or exploitable issues.

Limitation of Liability. To the maximum extent permitted by law, the Auditor and its affiliates, officers, employees, agents, and contractors shall not be liable for any indirect, incidental, special, consequential, or punitive damages, or for any loss of profits, revenue, data, or business opportunities, arising out of or related to the assessment, even if advised of the possibility of such damages. The Auditor's aggregate liability for direct damages is limited to the fees paid for the audit engagement.

Client Responsibility. The project owner/client retains sole responsibility for all decisions and actions taken in connection with the audited code, including fixing identified issues, deploying code to any environment, and applying security mitigations. The Auditor's recommendations are advisory; implementation and verification of fixes are the client's responsibility.

Third-Party Tools and Services. The Auditor may use third-party tools, services, or automated scanners as part of the assessment. Use of such tools is without warranty; the Auditor is not responsible for defects, failures, or inaccuracies arising from third-party services.

Confidentiality and Data Security. The Auditor will treat non-public client information as confidential. However, the Auditor cannot guarantee absolute security for data transmitted over the internet or third-party platforms. Client should take reasonable precautions to protect sensitive information.

Not Financial, Legal, or Investment Advice. The assessment is a technical security review only and is not financial, legal, tax, or investment advice. Clients should consult appropriate professionals for those matters.

Ownership and Governance Notes. If ownership is transferred, held by a multisig, or controlled by a governance contract, the client should document and disclose those arrangements; the Auditor's report reflects the ownership state observed during the engagement but may not reflect subsequent changes.

Governing Law and Counsel. This engagement and any disputes arising from it shall be governed by the laws agreed in the engagement terms. Clients are advised to seek legal counsel before relying on or publishing the report.

Acceptance. By proceeding with this engagement or using this report, the client acknowledges and accepts these terms.

